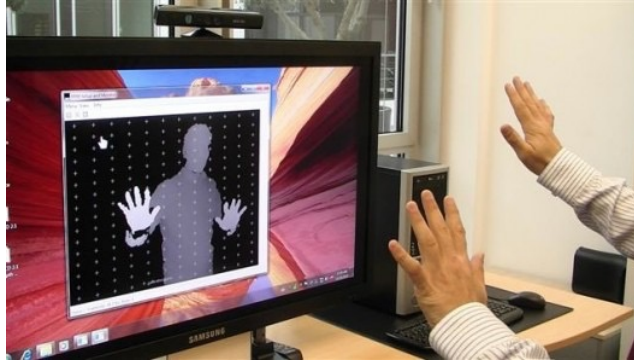
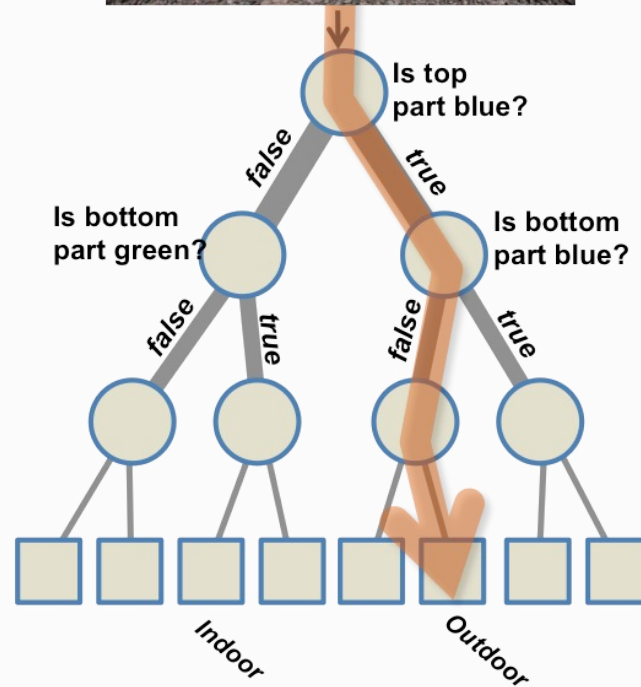




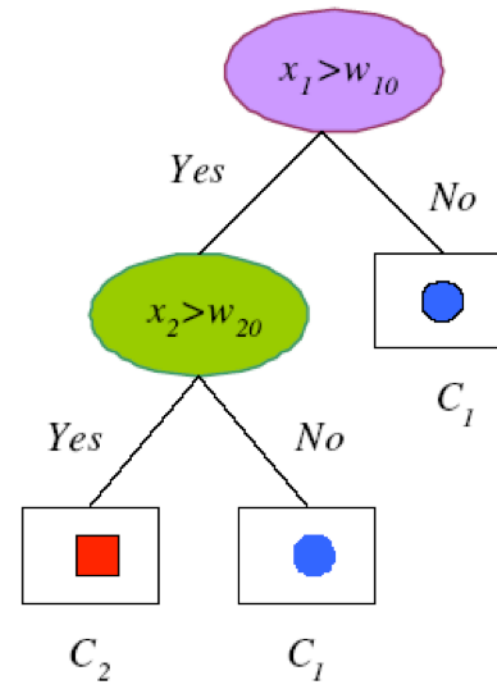
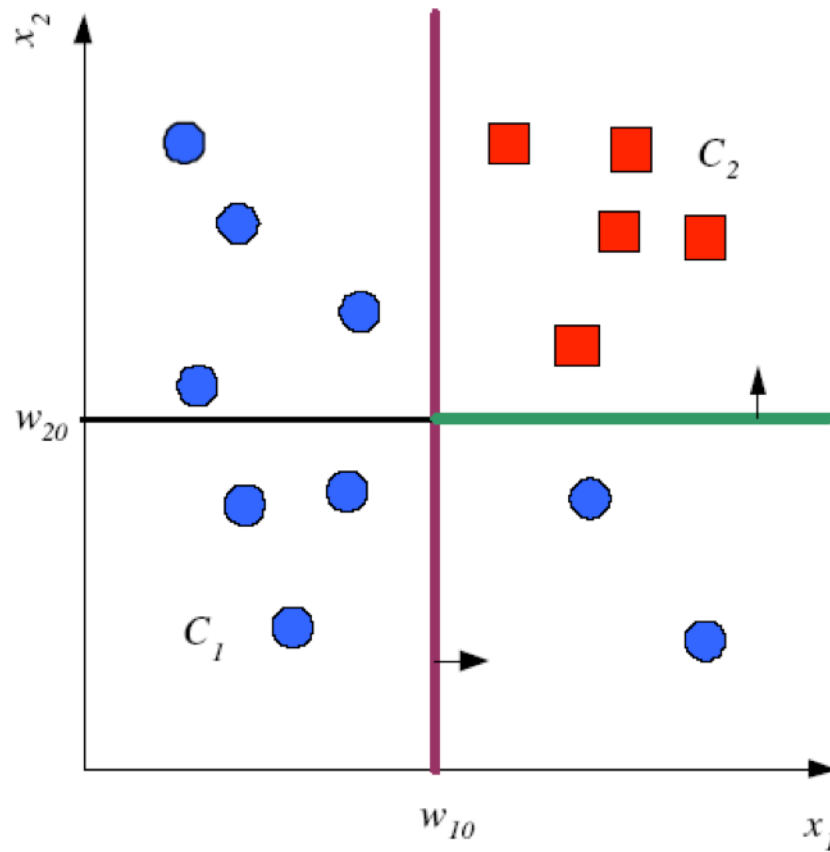
Decision Tree Learning

Decision Tree - Example

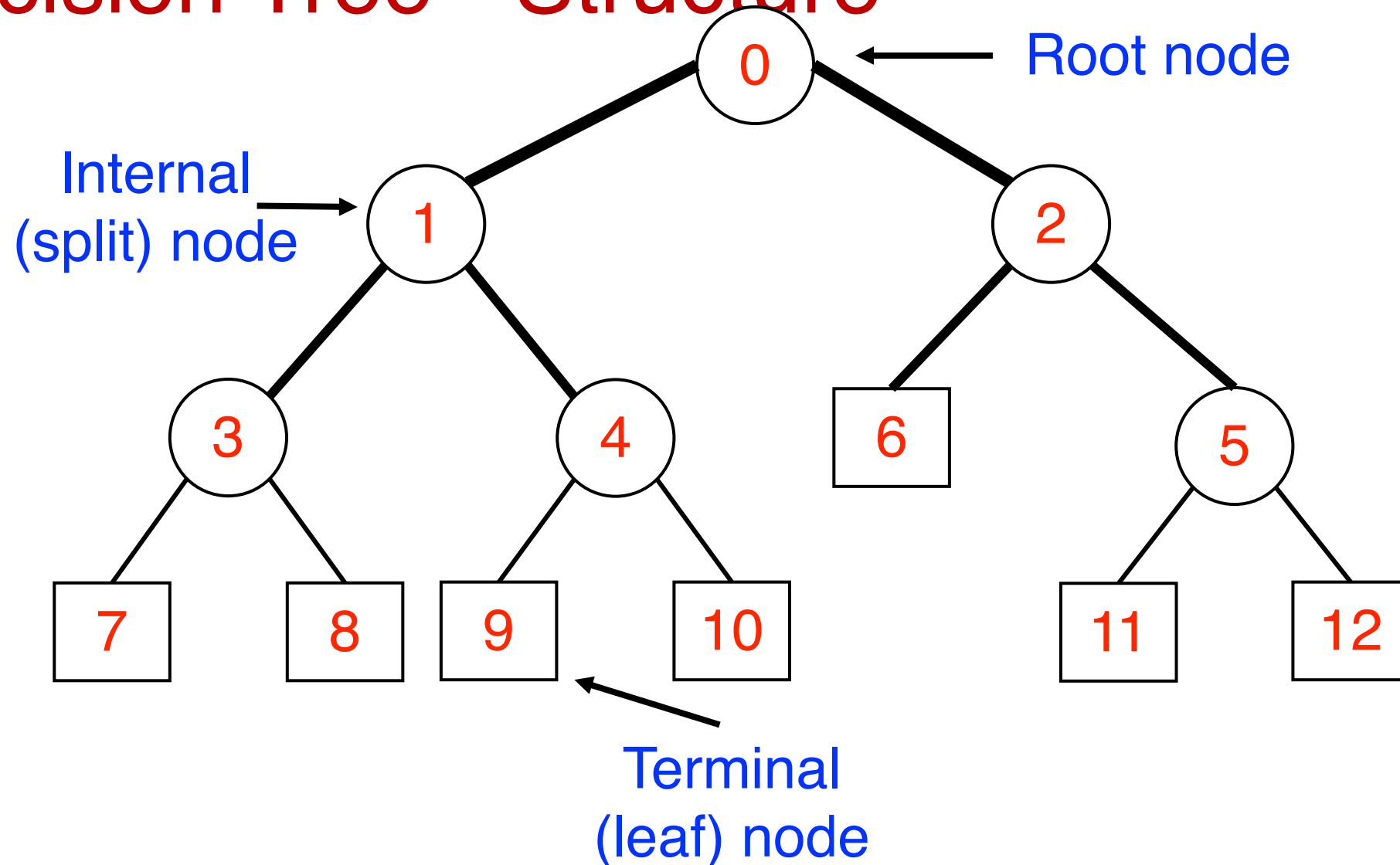
A decision tree



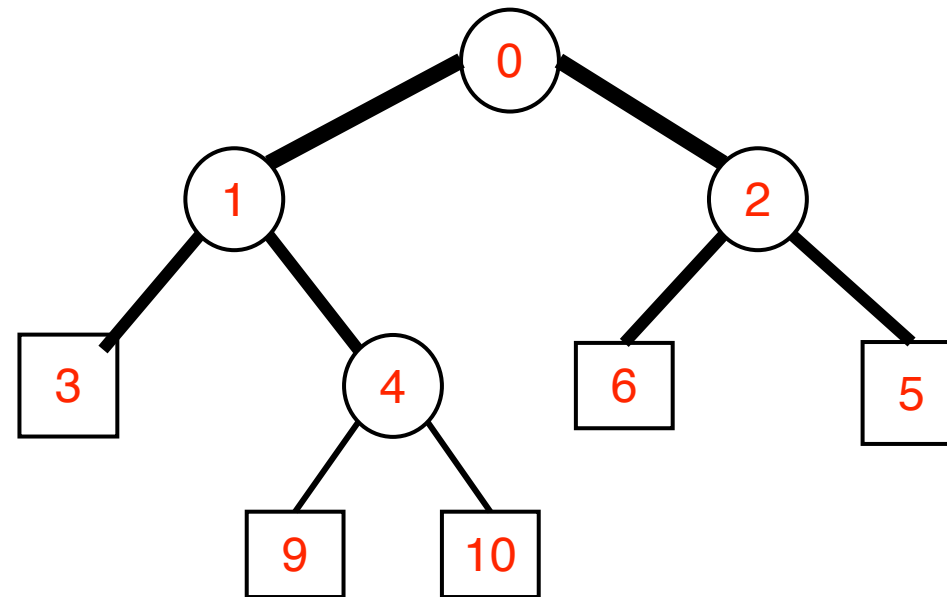
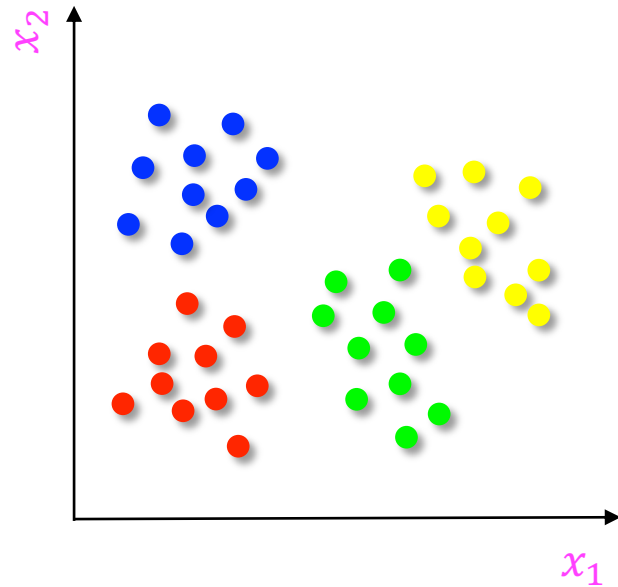
Classification Tree



Decision Tree - Structure



Learning Classification Tree

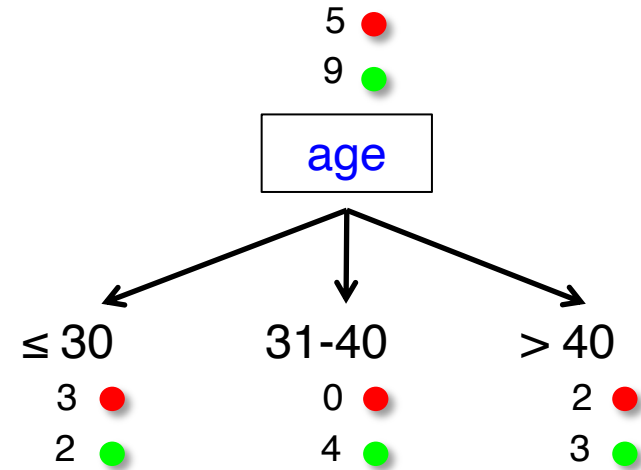


Example Dataset

example	age	income	student	Credit rating	Buys computer
x ₁	≤ 30	high	no	fair	no
x ₂	≤ 30	high	no	excellent	no
x ₃	31 – 40	high	no	fair	yes
x ₄	> 40	medium	no	fair	yes
x ₅	> 40	low	yes	fair	yes
x ₆	> 40	low	yes	excellent	no
x ₇	31 – 40	low	yes	excellent	yes
x ₈	≤ 30	medium	no	fair	no
x ₉	≤ 30	low	yes	fair	yes
x ₁₀	> 40	medium	yes	fair	yes
x ₁₁	≤ 30	medium	yes	excellent	yes
x ₁₂	31 – 40	medium	no	excellent	yes
x ₁₃	31 – 40	high	yes	fair	yes
x ₁₄	> 40	medium	no	excellent	no

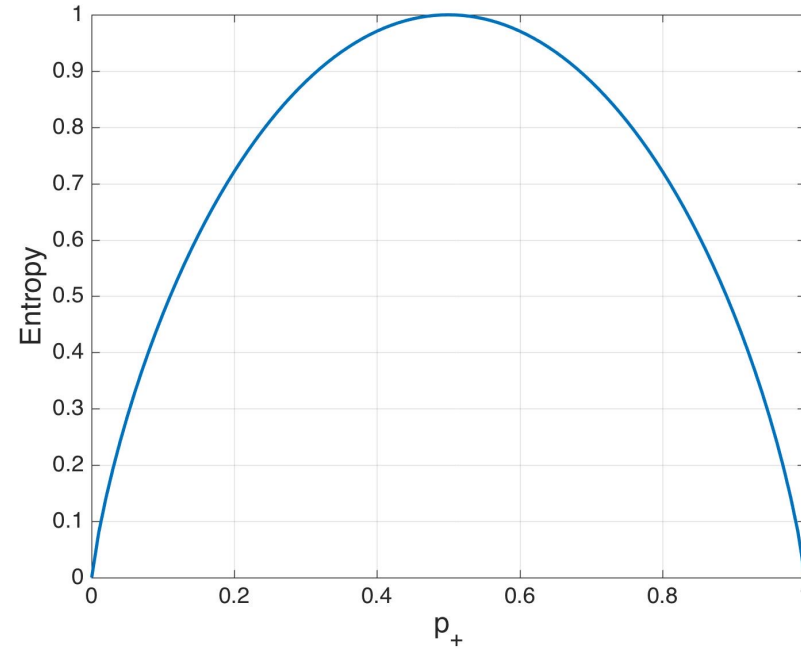
Tree Construction (1)

example	age	Buys computer
x ₁	≤ 30	no ●
x ₂	≤ 30	no ●
x ₈	≤ 30	no ●
x ₉	≤ 30	yes ●
x ₁₁	≤ 30	yes ●
x ₃	31 – 40	yes ●
x ₇	31 – 40	yes ●
x ₁₂	31 – 40	yes ●
x ₁₃	31 – 40	yes ●
x ₄	> 40	yes ●
x ₅	> 40	yes ●
x ₆	> 40	no ●
x ₁₀	> 40	yes ●
x ₁₄	> 40	no ●



Entropy - 2-class (1)

- I be the set of training examples
- p_+ be the proportion of positive examples in I
- p_- be the proportion of negative examples in I
- Entropy measure the impurity of I
$$\text{Entropy}(I) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$



Entropy - 2-class (2)

- Entropy(I) represents the expected number of bits to encode the class (+ or -) of a randomly drawn member of I
 - Derived from principles of Information Theory
 - Optimal length code assigns $-\log_2 p$ bits to a message having a probability p
 - The expected number of bits to encode + or - of a random member of I

$$\text{Entropy}(I) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$

Entropy - 2-class (2)

- Entropy(I) represents the expected number of bits to encode the class (+ or -) of a randomly drawn member of I
 - Derived from principles of Information Theory
 - Optimal length code assigns $-\log_2 p$ bits to a message having a probability p
 - The expected number of bits to encode + or - of a random member of I

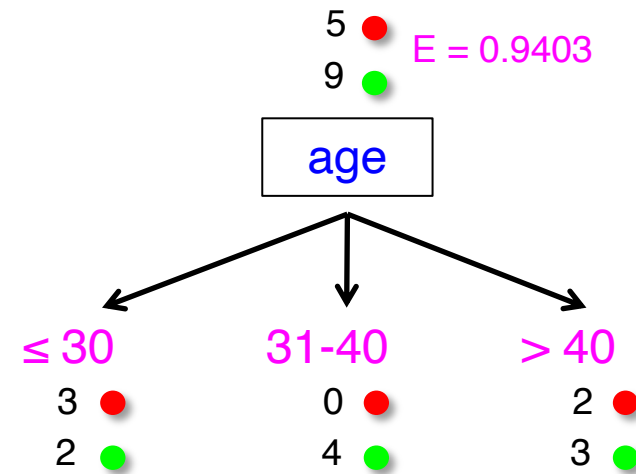
$$\text{Entropy}(I) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$

- Entropy if there are K classes?
- Sometimes H is used to refer to entropy

Tree Construction (2)

example	age	Buys computer
x ₁	≤ 30	no ●
x ₂	≤ 30	no ●
x ₈	≤ 30	no ●
x ₉	≤ 30	yes ●
x ₁₁	≤ 30	yes ●
x ₃	31 – 40	yes ●
x ₇	31 – 40	yes ●
x ₁₂	31 – 40	yes ●
x ₁₃	31 – 40	yes ●
x ₄	> 40	yes ●
x ₅	> 40	yes ●
x ₆	> 40	no ●
x ₁₀	> 40	yes ●
x ₁₄	> 40	no ●

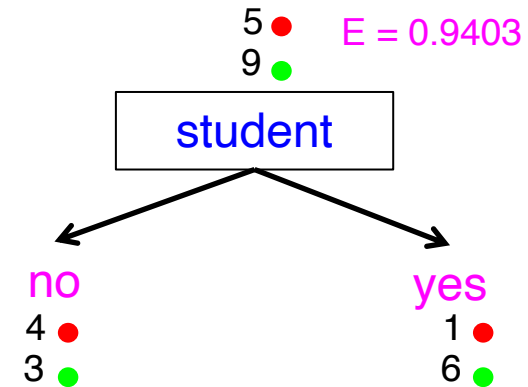
• Entropy for Age



Tree Construction (3)

example	student	Buys computer
x ₁	no	no ●
x ₂	no	no ●
x ₃	no	yes ●
x ₄	no	yes ●
x ₈	no	no ●
x ₁₂	no	yes ●
x ₁₄	no	no ●
x ₅	yes	yes ●
x ₆	yes	no ●
x ₇	yes	yes ●
x ₉	yes	yes ●
x ₁₀	yes	yes ●
x ₁₁	yes	yes ●
x ₁₃	yes	yes ●

• Entropy for Student



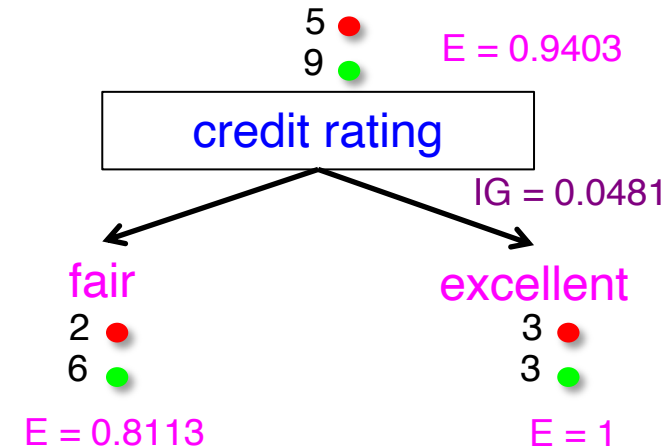
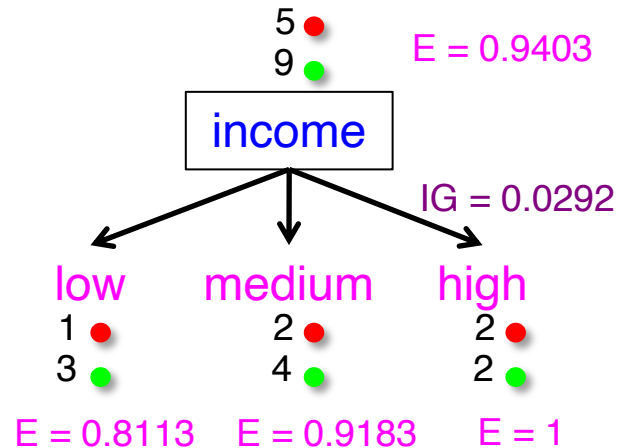
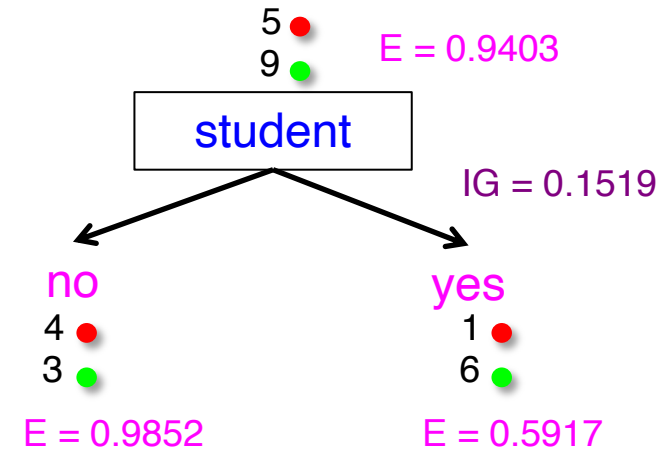
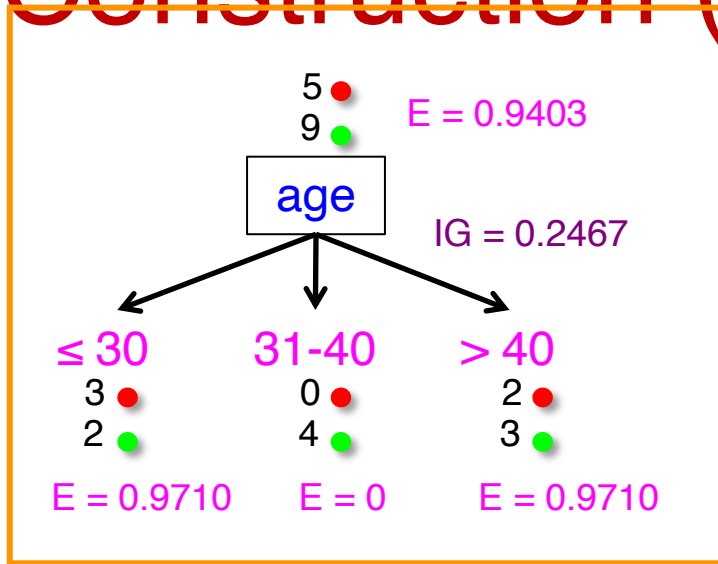
Information Gain

- $\text{Gain}(I, x_i)$ – expected reduction in entropy due to sorting on attribute x_i

$$\text{Gain}(I, x_i) = \text{Entropy}(I) - \sum_{v \in \text{Values}(x_i)} \frac{|I_v|}{|I|} \text{Entropy}(I_v)$$

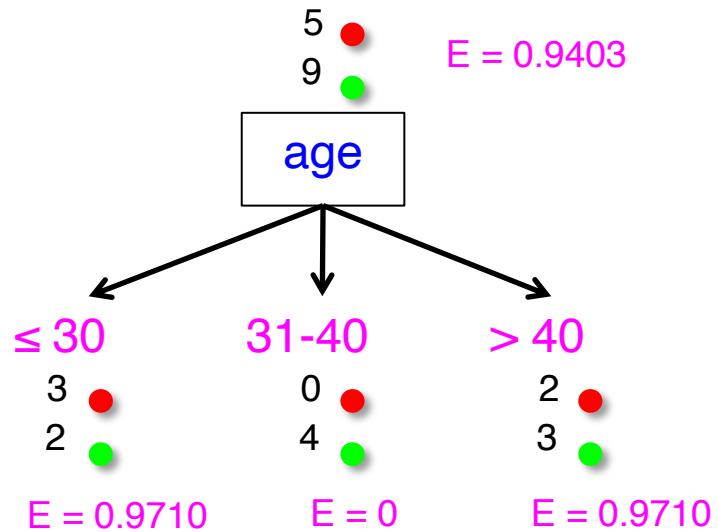
- $\text{Values}(x_i)$ – all possible values that attribute x_i can take.
- $I_v \subset I$ – data points in I that take the value v for the attribute x_i

Tree Construction (4)



Tree Construction (5)

- Selecting the next split

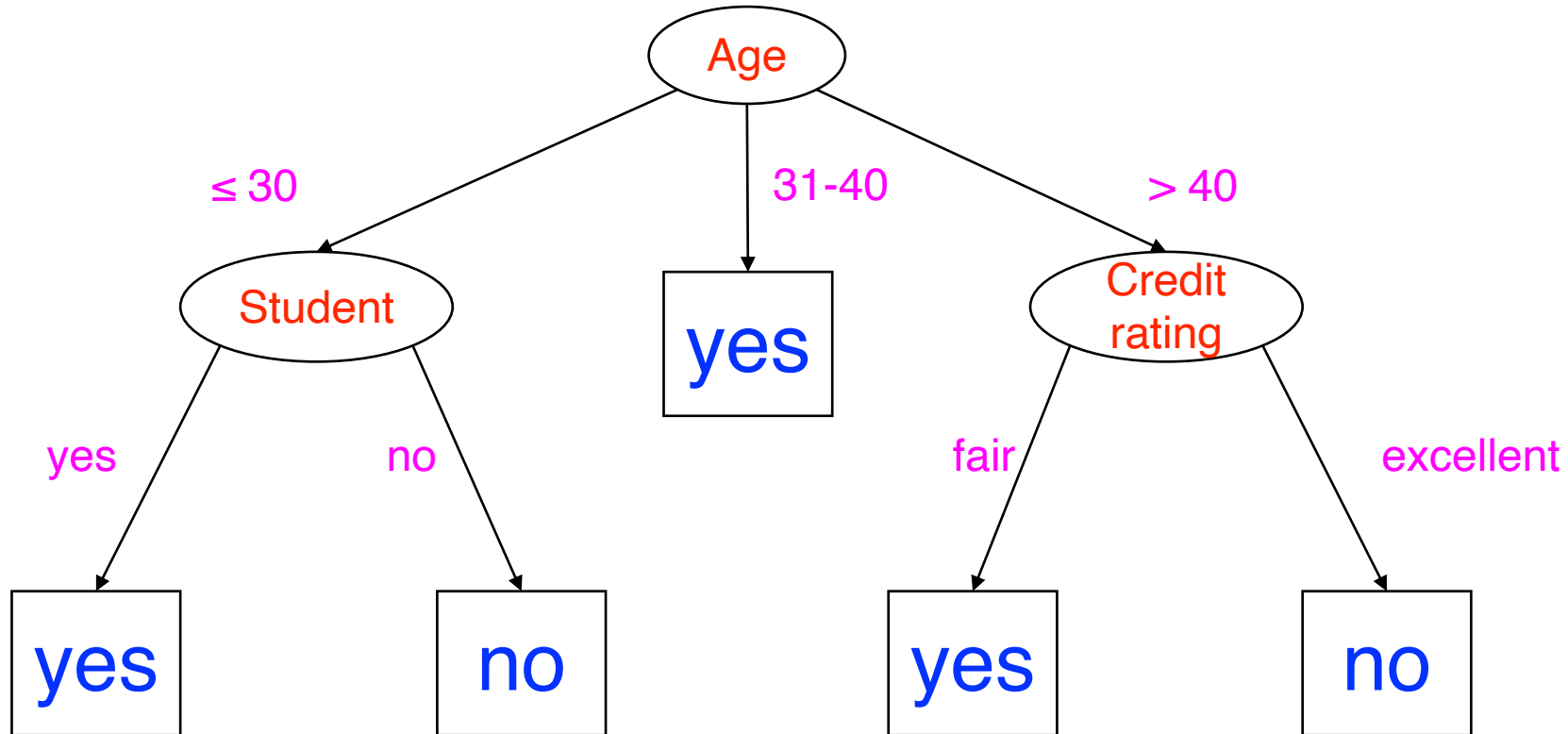


example	income	student	Credit rating	Buys computer
x_1	high	no	fair	no
x_2	high	no	excellent	no
x_8	medium	no	fair	no
x_9	low	yes	fair	yes
x_{11}	medium	yes	excellent	yes

Putting it all together – ID3 Algorithm

- ID3(Examples, Target – Attribute, Attributes):
- Create a **Root** node for the tree
- If all the Examples are positive, return single-node tree **Root**, with **positive label**
- If all the Examples are negative, return single-node tree **Root**, with **negative label**
- Otherwise Begin
 - A – attribute from attributes that best classified Examples
 - The decision attribute for the **Root** is A
 - For each possible value v for attribute A
 - Add a new tree branch below **Root**, corresponding to the test $A = v$
 - Let Examples $_v$ be the subset of Examples that have v as the value for attribute A
 - If Examples $_v$ is empty
 - Then below this new branch add a leaf node with the most frequently occurring Target – Attribute in Examples
 - Else, below this new branch add the subtree ID3(Examples $_v$, Target – Attribute, Attributes – { A })

A Decision Tree for the Dataset

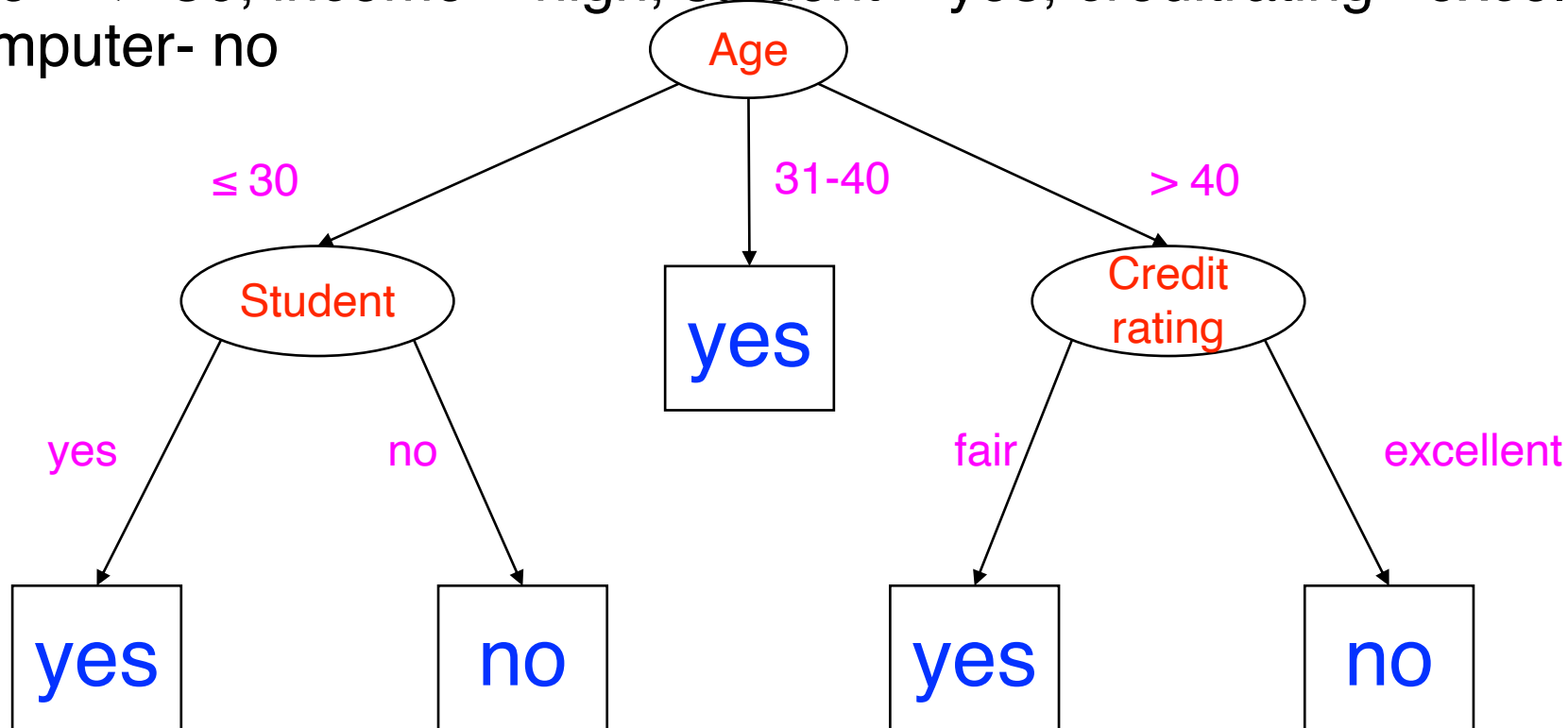


Analyzing Decision Trees

- Hypothesis Space - Growing
 - The space is complete – any discrete valued function can be represented as a decision tree, relative to the available attributes
 - Target function is surely present in the space search
- Outputs only a single hypothesis
 - Consistent with the training set
 - Cannot determine alternate decision trees that are consistent with the training set
 - Cannot resolve between competing hypotheses
- No backtracking
 - Greedy algorithm
- Statistically-based search choices
 - Use all 'available' training data to make decisions on how to refine the current hypothesis

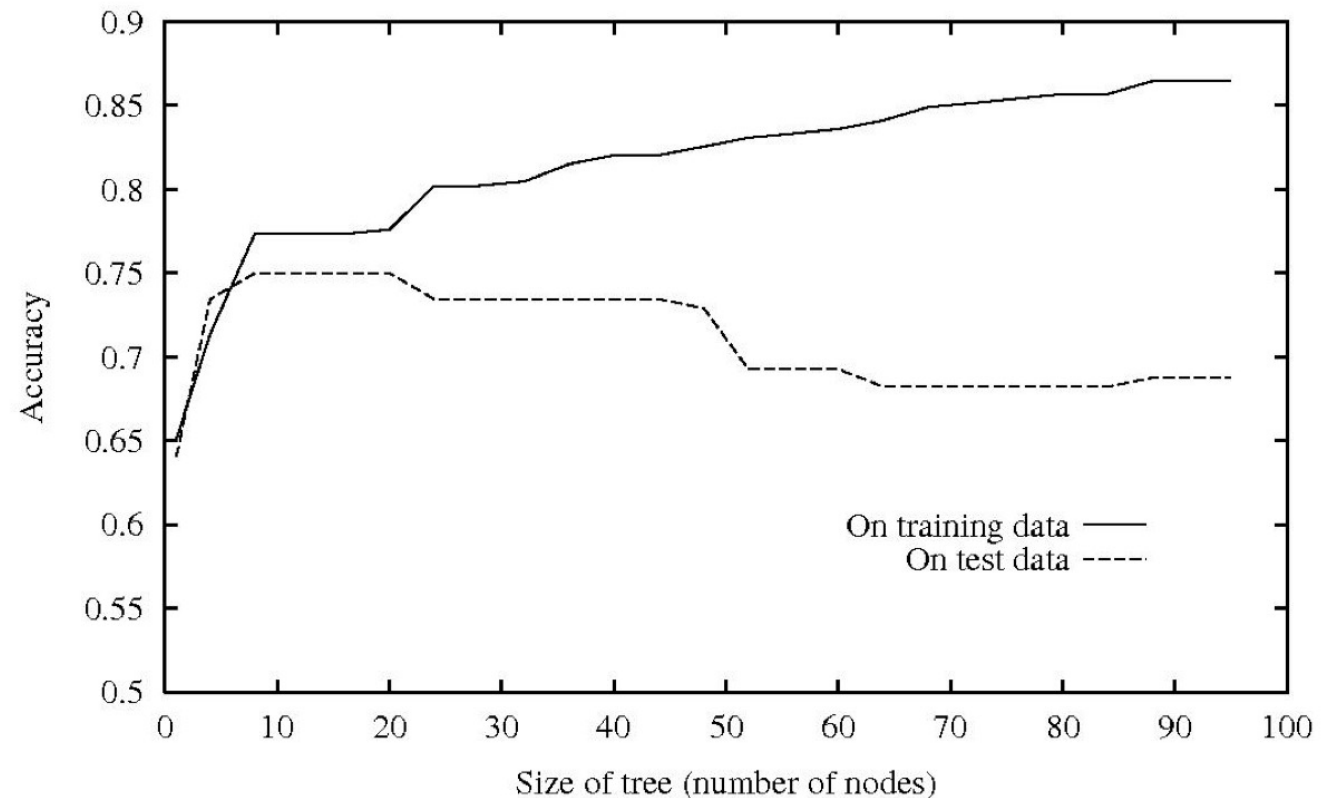
Overfitting in Decision Trees (1)

- Consider adding a noisy training example
 - Age - ≤ 30 , income – high, student – yes, creditrating –excellent, buys computer- no



Overfitting in Decision Trees (2)

- ID3 - applied to the task of learning which medical patients have a form of diabetes



Overfitting – Formal Definition

- Consider the error of hypothesis h over
 - Training data – $E(h|X)$
 - Entire distribution P of the data – $E_P(h)$
- Hypothesis $h \in H$ overfits the training data if there is an alternative hypothesis $h' \in H$ such that
 - $E(h|X) < E(h'|X)$
 - $E_P(h) > E_P(h')$

Avoiding Overfitting (1)

- Two (three) strategies to prevent overfitting
 - Stop growing when data split is not statistically significant
 - Grow full tree, then post prune
 - *Using ensembles – Random/Decision Forests*
- How to select “best” tree
 - Measure performance over training data
 - Measure performance over separate validation data
 - Add complexity penalty to performance measure

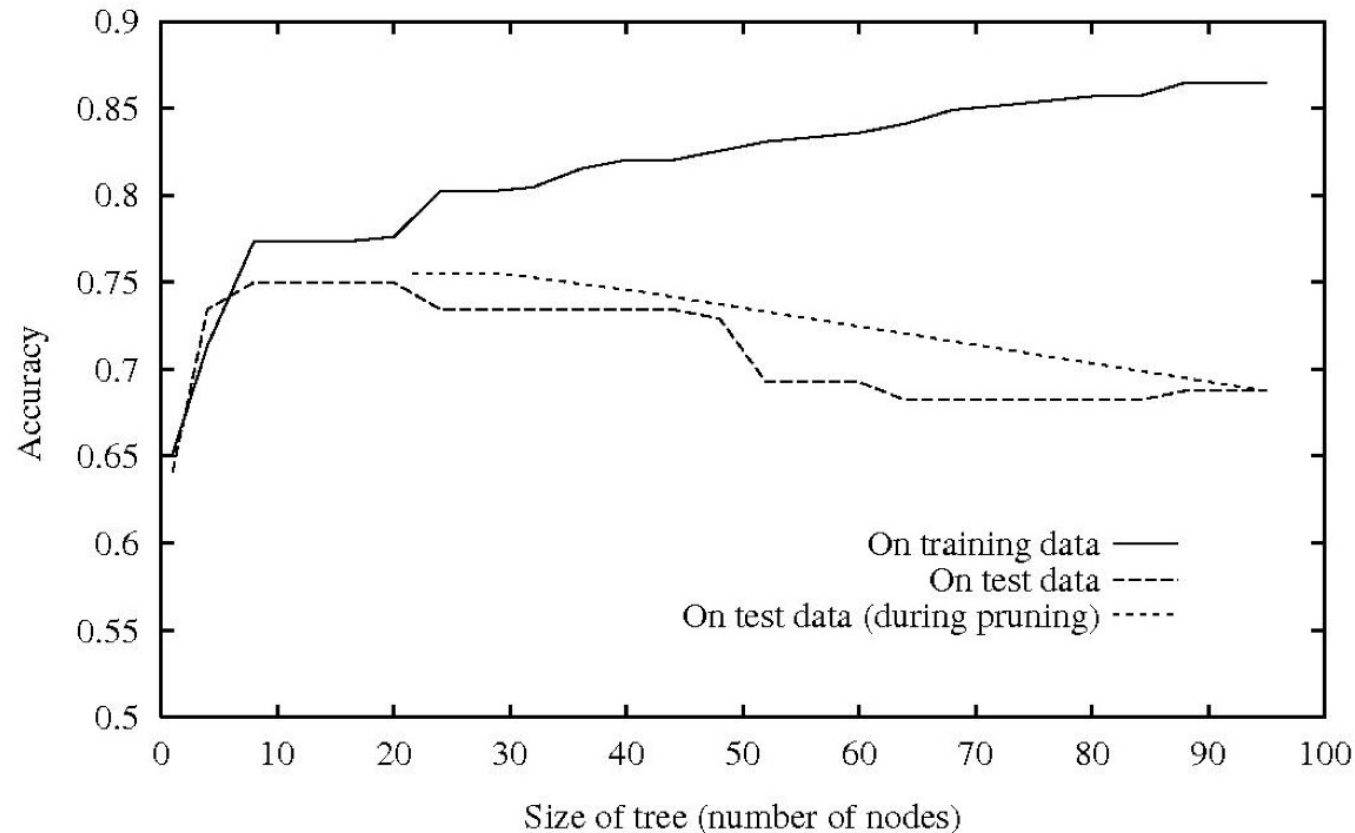
Avoiding Overfitting (2)

- Reduced Error Pruning

- Split data into training and validation set
- Do until further pruning is harmful
 - Evaluate the impact on the validation set of pruning each possible subtree (node, plus those below it)
 - Greedily remove the subtree that results in maximum improvement over the validation set
- Produces the smallest version of the most accurate subtree

Avoiding Overfitting (3)

- Reduced Error Pruning - Impact



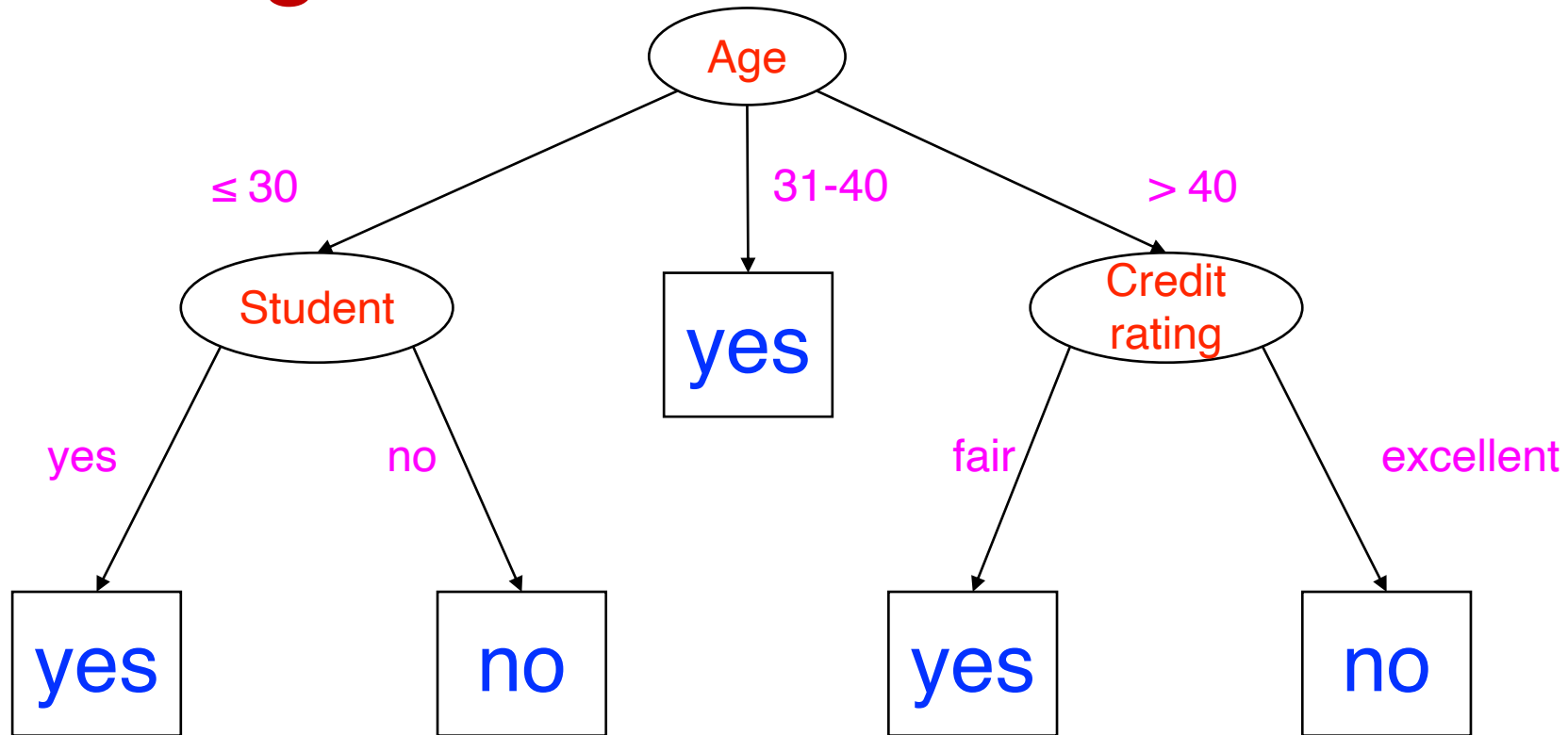
Avoiding Overfitting (4)

- Rule Post-Pruning

- Infer the decision tree from the training set
- Convert the learned tree into an equivalent set of rules
- Prune each rule by removing any preconditions that result in improving estimated performance
 - Check the preconditions against an held out validation set
- Sort the pruned rules by their estimated performance

- Most frequently used method (C4.5)

Converting a Decision Tree to Rules



Miscellaneous Issues in Decision Tree Learning (1)

- Real-Valued Features

- Quantization – creating a discrete feature set to test real values
 - Example – credit rating = 820, new feature – credit rating >745
- Sort the values and try splits on mid points

Credit rating	189	245	256	370	377	394	605	720
Buys computer	no	no	yes	no	yes	yes	no	no

Miscellaneous Issues in Decision Tree Learning (2)

- Alternative measures for selecting attributes
 - Problem with information gain

Miscellaneous Issues in Decision Tree Learning (3)

example	age	Buys computer
x ₁	20	no
x ₂	21	no
x ₃	23	yes
x ₄	34	yes
x ₅	25	yes
x ₆	41	no
x ₇	37	yes
x ₈	18	no
x ₉	27	yes
x ₁₀	45	yes
x ₁₁	29	yes
x ₁₂	35	yes
x ₁₃	36	yes
x ₁₄	46	no

Miscellaneous Issues in Decision Tree Learning (4)

- Alternative measures for selecting attributes
 - Problem with information gain
- *Gain Ratio* – incorporates a term called split information that is sensitive to how broadly and uniformly the attribute splits the data

$$\text{SplitInformation}(I, x_i) = - \sum_{c=1}^C \frac{|I_c|}{|I|} \log_2 \frac{|I_c|}{|I|}, C = |\text{Values}(x_i)|$$

$$\text{Gain Ratio}(I, x_i) = \frac{\text{Information Gain}(I, x_i)}{\text{Split Information}(I, x_i)}$$

Exercise – Compute the Gain Ratio for Age attribute

Miscellaneous Issues in Decision Tree Learning (5)

- Alternative measures for selecting attributes
 - Problem with information gain
 - *Gain Ratio*
- *Cost sensitive information gain*
 - Applications where each attribute has an associated cost
 - Medical Diagnosis – MRI Scan costs ₹ 5000
 - Goal is to learn a consistent tree with low expected cost
 - Modify information gain to include cost

$$\frac{\text{Information Gain}(I, x_i)}{\text{Cost}(x_i)}$$

Miscellaneous Issues in Decision Tree Learning (6)

- Handling Training Examples with Missing Attribute values
- Node n tests the attribute x_i
 - Assign the most common value of x_i among other examples at node n
 - Assign the most common value of x_i among other examples with the same target value at node n
 - Assign probability p_v to each possible value v of x_i
 - Assign fraction p_v of the example to each descendant from the node

Advantages and Disadvantages of Trees

Easy to explain – results in set of rules

Closely mirrors human decision making process

Visualization is easy for interpretation even by a non-machine learning expert.

Number of hyper-parameters to be tuned is almost none

Trees are prone to overfitting

Prediction accuracies are low compared to other approaches

- Aggregation of many decision trees can substantially improve the performance

Summary

- Popular concept learning method
- Uses information theoretic heuristic for the construction of the tree
- The model is completely expressive
- Inductive Bias is towards shorter trees

Markov Decision Processes

CHAPTER 17

Slides have been adopted from Dan Klein and Dieter Fox

Example: Grid World

❑ A maze-like problem

- The agent lives in a grid
- Walls block the agent's path

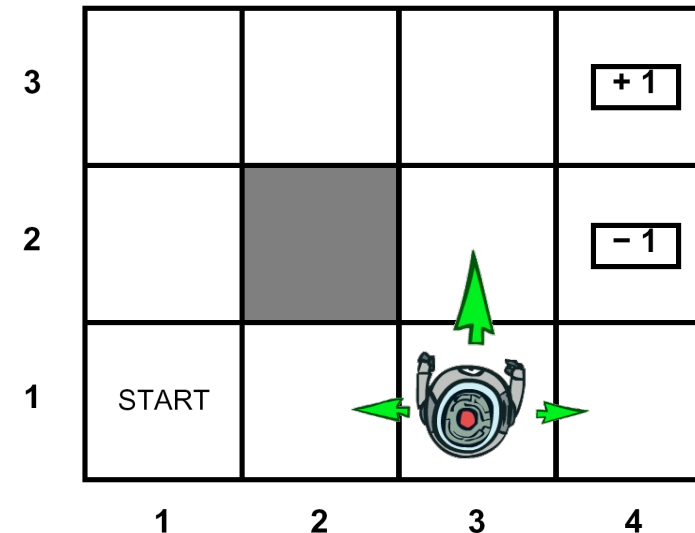
❑ Noisy movement: actions do not always go as planned

- 80% of the time, the action North takes the agent North (if there is no wall there)
- 10% of the time, North takes the agent West; 10% East
- If there is a wall in the direction the agent would have been taken, the agent stays put

❑ The agent receives rewards each time step

- Small “living” reward each step (can be negative)
- Big rewards come at the end (good or bad)

❑ Goal: maximize sum of rewards



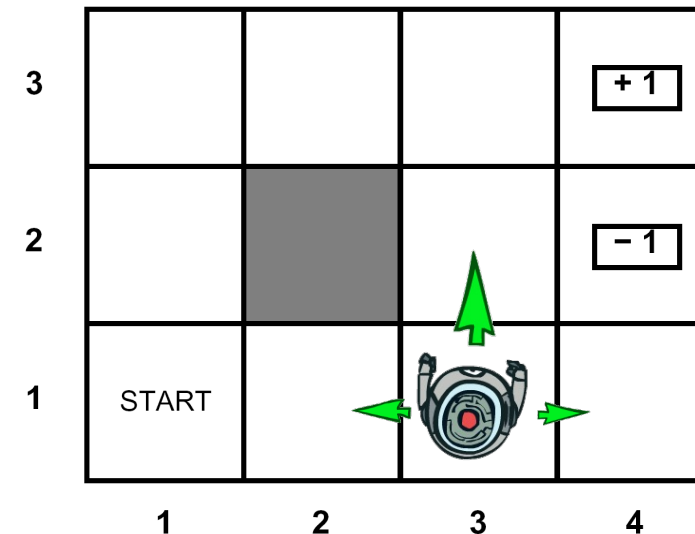
Markov Decision Processes -Elements

□ An MDP is defined by:

- A set of states s - S
- A set of actions a - A
- A transition function $T(s, a, s')$
 - Probability that a from s leads to s' , i.e., $P(s'|s, a)$
 - Also called the model or the dynamics
- A reward function $R(s, a, s')$
 - Sometimes just $R(s)$ or $R(s')$
- A start state
- Maybe a terminal state

□ MDPs are non-deterministic search problems

- One way to solve them is with expectimax search



What is Markov about MDPs?

- ❑ “Markov” generally means that given the present state, the future and the past are independent
- ❑ For Markov decision processes, “Markov” means action outcomes depend only on the current state

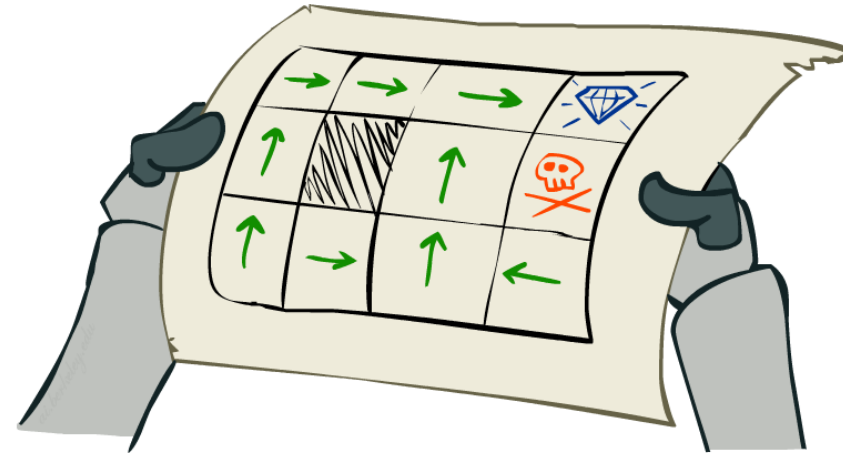
$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t, \dots, S_0 = s_0) = \\ P(S_{t+1} = s' | S_t = s_t, A_t = a_t)$$



- ❑ This is just like search, where the successor function could only depend on the current state (not the history)

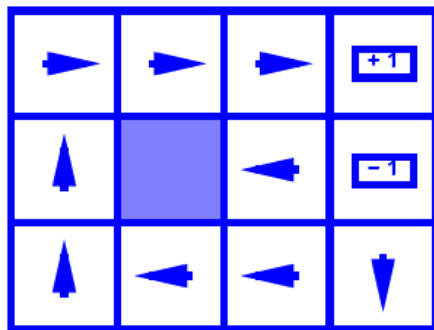
Policies

- ❑ In deterministic single-agent search problems, we wanted an optimal plan, or sequence of actions, from start to a goal
- ❑ For MDPs, we want an optimal policy $\pi^*: S \rightarrow A$
 - A policy gives an action for each state
 - An optimal policy is one that maximizes expected utility if followed
 - An explicit policy defines a reflex agent
- ❑ Expectimax didn't compute entire policies
 - It computed the action for a single state only

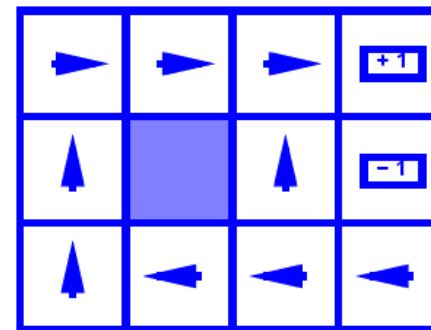


Optimal policy when $R(s, a, s') = -0.03$
for all non-terminals s

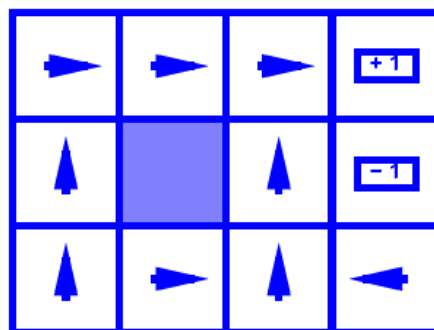
Optimal Policies



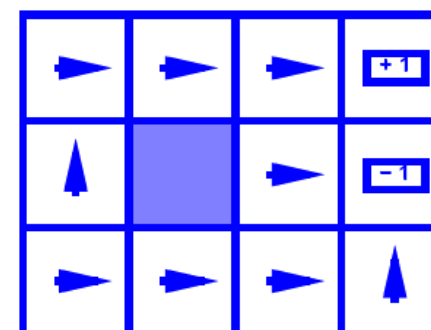
$$R(s) = -0.01$$



$$R(s) = -0.03$$



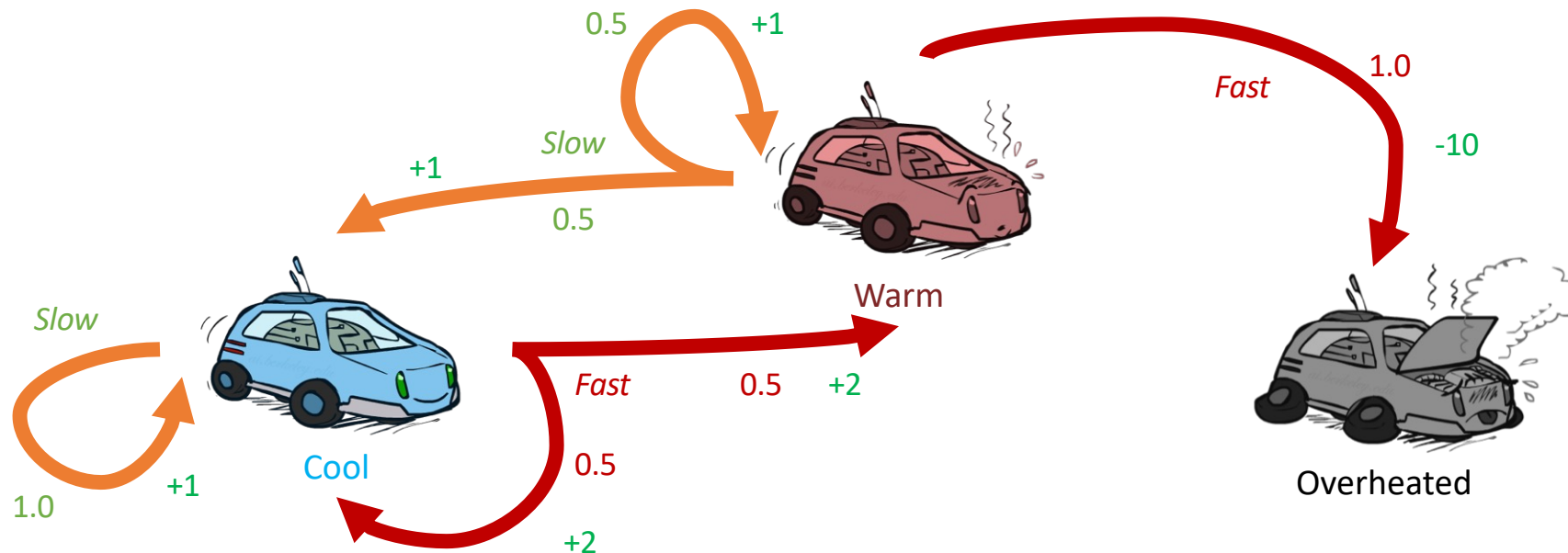
$$R(s) = -0.4$$



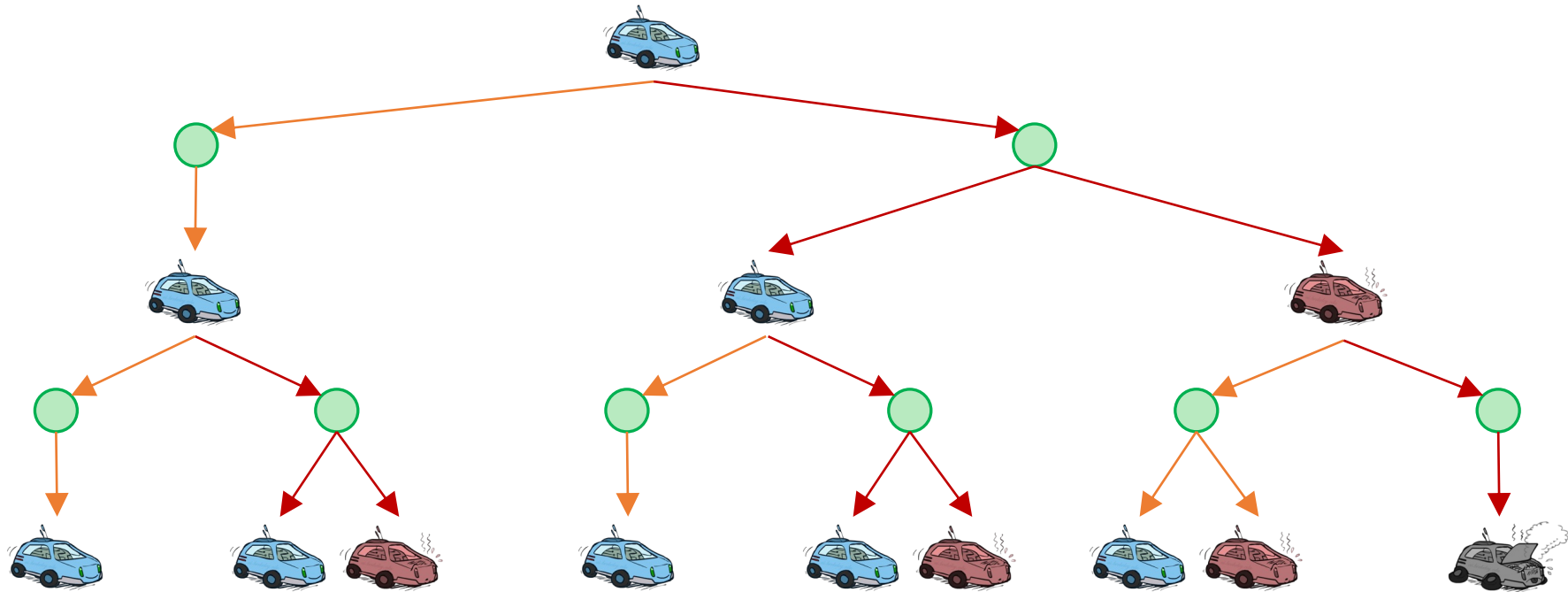
$$R(s) = -2.0$$

Example: Racing

- ❑ A robot car wants to travel far, quickly
- ❑ Three states: Cool, Warm, Overheated
- ❑ Two actions: Slow, Fast
- ❑ Going faster gets double reward

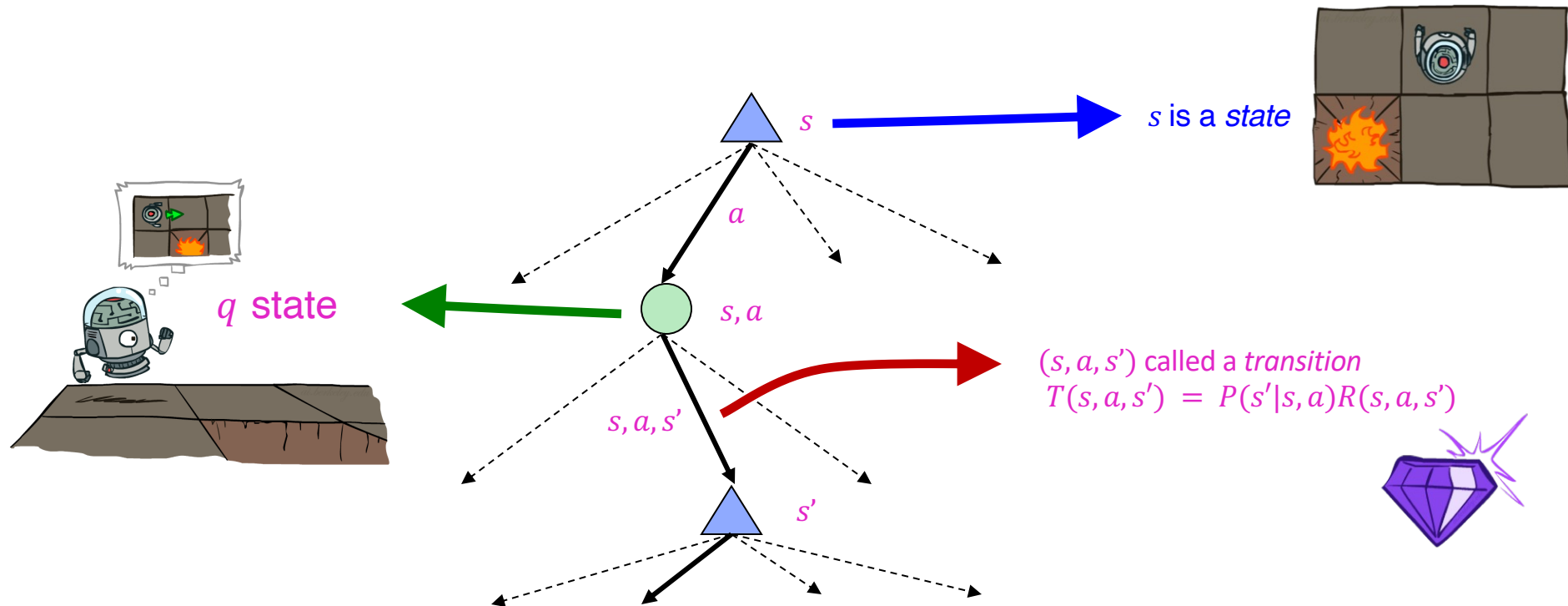


Racing Search Tree



MDP Search Trees

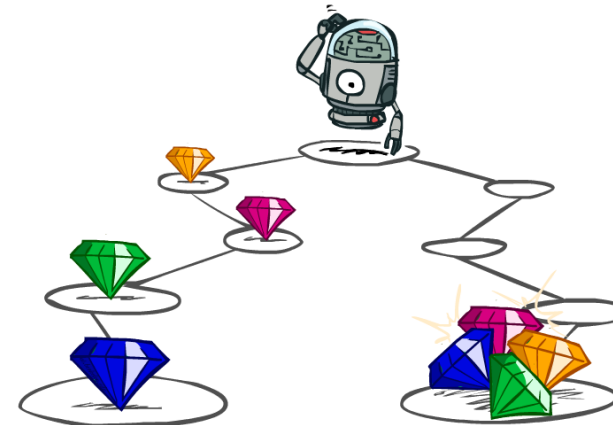
- Each MDP state projects an expectimax-like search tree



Utilities of Sequences/ over Time

□ What preferences should an agent have over reward sequences?

- More or less? $[1, 2, 3]$ or $[2, 3, 4]$
- Now or later? $[0, 0, 1]$ or $[1, 0, 0]$



Discounting

- ❑ It is reasonable to maximize the sum of rewards
- ❑ It is also reasonable to prefer rewards now to rewards later
- ❑ One solution: values of rewards decay exponentially



1

Worth Now



γ

Worth Next Step



γ^2

Worth In Two Steps

Discounting (2)

❑ How to discount?

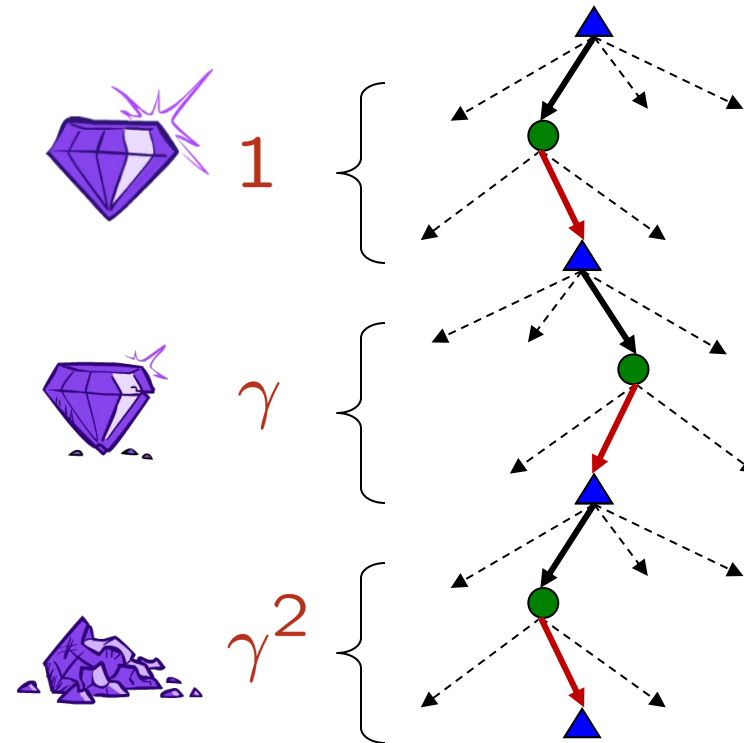
- Each time we descend a level, we multiply in the discount once

❑ Why discount?

- Sooner rewards probably do have higher utility than later rewards
- Also helps our algorithms converge

❑ Example: discount of 0.5

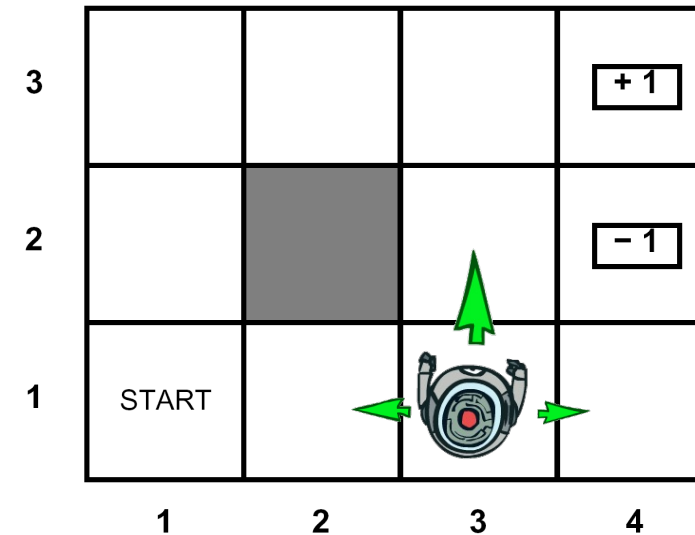
- $U([1,2,3]) = 1*1 + 0.5*2 + 0.25*3$
- $U([1,2,3]) < U([3,2,1])$



Recap - Markov Decision Processes - Elements

□ An MDP is defined by:

- A set of states s - S
- A set of actions a - A
- A transition function $T(s, a, s')$
 - Probability that a from s leads to s' , i.e., $P(s' | s, a)$
 - Also called the model or the dynamics
- A reward function $R(s, a, s')$
 - Sometimes just $R(s)$ or $R(s')$
- A start state
- Maybe a terminal state



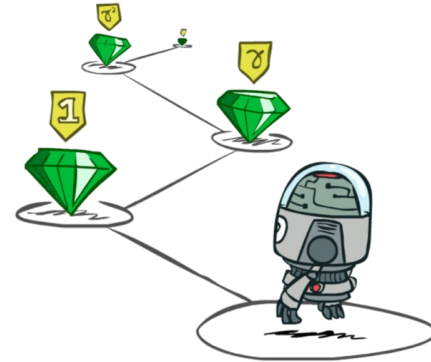
□ Policies

□ Stationary Preferences

Stationary Preferences

□ Theorem: if we assume stationary preferences:

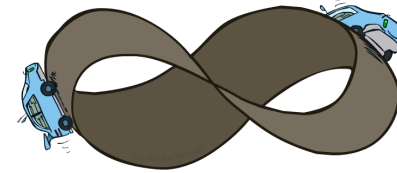
$$\begin{aligned} [a_1, a_2, \dots] &> [b_1, b_2, \dots] \\ &\iff \\ [r, a_1, a_2, \dots] &> [r, b_1, b_2, \dots] \end{aligned}$$



□ Then: there are only two ways to define utilities

- Additive utility: $U([r_0, r_1, \dots]) = r_0 + r_1 + r_2 + \dots$
- Discounted utility: $U([r_0, r_1, \dots]) = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$

Utilities Over Time



❑ Problem: What if the game lasts forever? Do we get infinite rewards?

❑ Solutions:

- Finite horizon: (similar to depth-limited search)

- Terminate episodes after a fixed T steps
- Gives nonstationary policies (π depends on time left)

- Discounting: use $0 < \gamma < 1$

$$U([r_0, r_1, \dots]) = \sum_{t=0}^{\infty} \gamma^t r_t \leq R_m / (1 - \gamma)$$

- Smaller γ means smaller “horizon” – shorter term focus
- Absorbing state: guarantee that for every policy, a terminal state will eventually be reached (like “overheated” for racing)

Discounting (3)

□ Given:

10				1
a	b	c	d	e

- Actions: East, West, and Exit (only available in exit states a, e)
- Transitions: deterministic

□ Q 1: For $\gamma = 1$, what is the optimal policy?

□ Q2: For $\gamma = 0.1$, what is the optimal policy?

□ Q3: For which γ are West and East equally good when in state d?

Solving MDPs

